

Лабораторна робота №1

Тема: Розгортання проекту. Створення сторінок. Стилiзацiя

Мета роботи: навчитися розгортати проект; створювати сторінки; стилізувати компоненти.

Практичні завдання

Завдання 1: Розгорнути проект

Для створення базової структури(рис.1) використано утиліту create-next-app. Було обрано структуру з використанням директорії src, що є стандартом для професійної розробки.

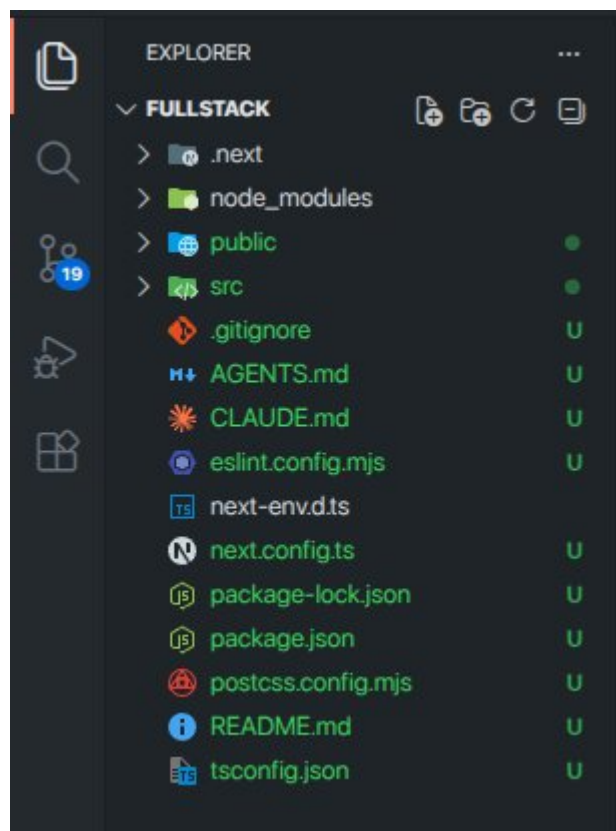


Рисунок 1 – Структура проекту.

Завдання 2: Створіть сторінки

Створення директорії src/app/(inner): Використано механізм Route Groups (папка в дужках), щоб об'єднати внутрішні сторінки спільним лейаутом, не

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 – Лр.1							
Змн.	Арк.	№ докум.	Підпис	Дата								
Розроб.		Маковський М.С			Звіт з лабораторної роботи №1				Літ.	Арк.	Акрушіє	
Перевір.		Фант М.О									1	8
Реценз.									ФІКТ, гр. ВТ-23-2			
Н. Контр.												
Зав.каф.												

змінюючи їхні URL-адреси

У корені `src/app/` створено головну сторінку `page.tsx`

У папці `(inner)` створено дерево каталогів для маршрутів: `/articles`, `/articles/favorite`, `/articles/create`, `/profile/settings` та `/profile/security`. У кожній кінцевій папці розміщено файл `page.tsx`

`src/app/(inner)/layout.tsx`: Реалізовано основний Layout для всіх внутрішніх сторінок. У нього додано навігаційну панель із посиланнями на головні розділи: Статті та Профіль.

`src/app/(inner)/articles/layout.tsx`: суто для розділу статей. Він автоматично відображає додаткове меню (Обране/Створити) лише для сторінок, що починаються з `/articles`

`src/components/NavLink.tsx`: Створено окремий клієнтський компонент для навігації. Він використовує хук `usePathname` для перевірки поточного маршруту та динамічного додавання класів Tailwind CSS для підсвічування активного пункту меню.

У всіх файлах використано Absolute Imports через аліас `@/*`, що дозволяє коректно звертатися до компонентів незалежно від рівня вкладеності файлу.

Оновлену структуру проєкту зображено на рис.2

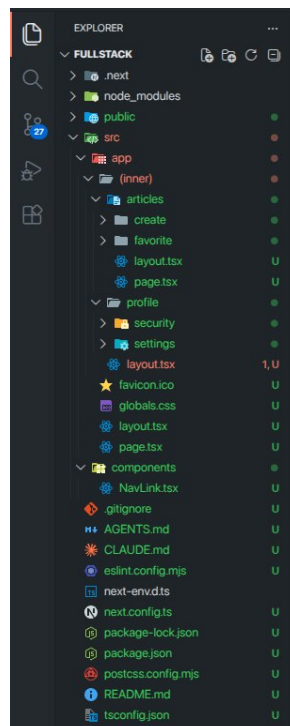


Рисунок 2 – Оновлена структура проєкту після виконання завдання 2.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.1	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

Завдання 3: Отримання даних

Для отримання списку статей(рис.3) було використано Server Components. У файлі src/app/articles/page.tsx реалізовано асинхронну функцію для виконання запиту GET до JSONPlaceholder. Для виконання пункту 2 було створено спеціальний файл loading.tsx у тій же директорії. Next.js автоматично відображає цей компонент (скелетон або індикатор завантаження), поки триває запит до API.

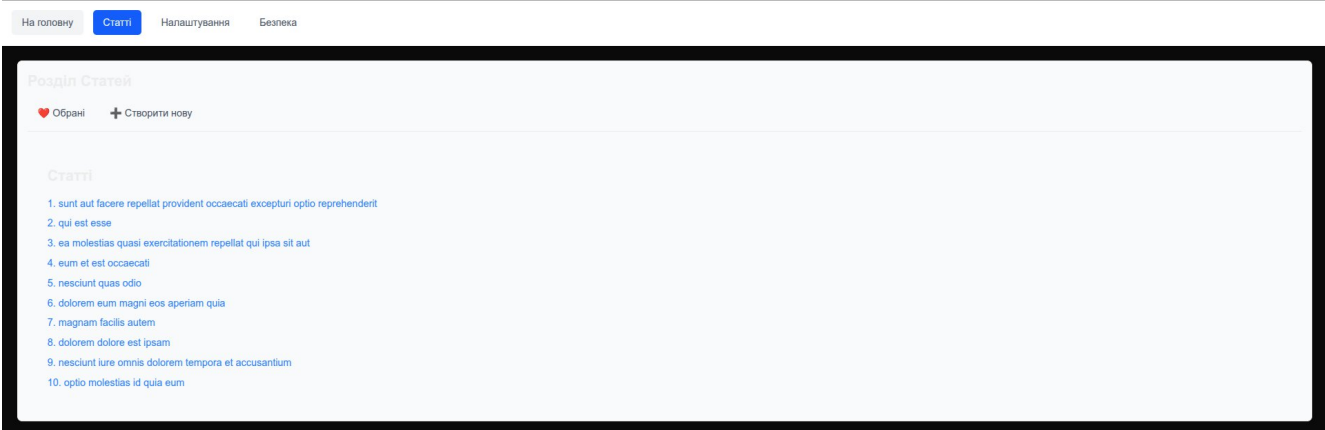


Рисунок 3 – Список статей.

Для сторінки /articles/favorite було розроблено окремий клієнтський компонент FavoriteArticle.tsx. Кожен екземпляр компонента самостійно виконує запит до API за певним id. Це дозволило реалізувати незалежні індикатори завантаження(рис.4): якщо один запит виконується швидше за інші, користувач одразу бачить контент цієї статті, поки інші ще завантажуються.

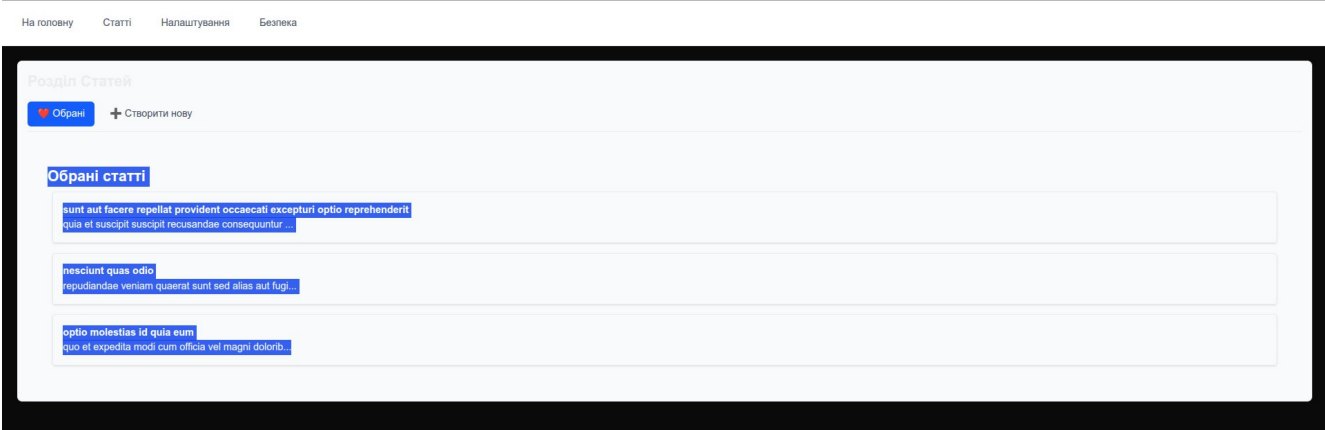


Рисунок 4 – Список улюблених статей.

Створено динамічний маршрут `src/app/articles/[id]/page.tsx`. На цій сторінці(рис.5) реалізовано паралельне отримання даних (Parallel Data Fetching) за допомогою `Promise.all`. Це дозволяє одночасно завантажувати дані про саму статтю та коментарі до неї (`/posts/{id}/comments`), що скорочує загальний час очікування.

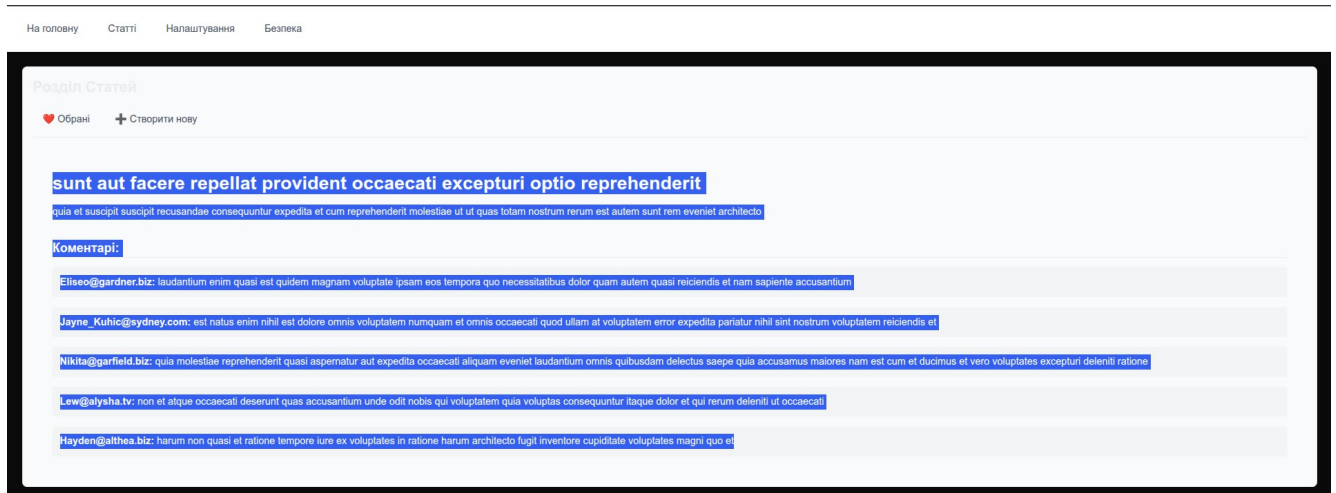


Рисунок 5 – Детальна сторінка статті з коментарями.

Згідно з пунктом 6, сторінку детального перегляду було оптимізовано за допомогою функції `generateStaticParams`. Це перетворило динамічний маршрут на статично генерований (SSG)(рис.7) для статей з ID від 1 до 10. Під час виконання команди `npm run build` Next.js заздалегідь створив HTML-файли для цих десяти постів, що забезпечує миттєве завантаження без звернення до API в момент кліку.

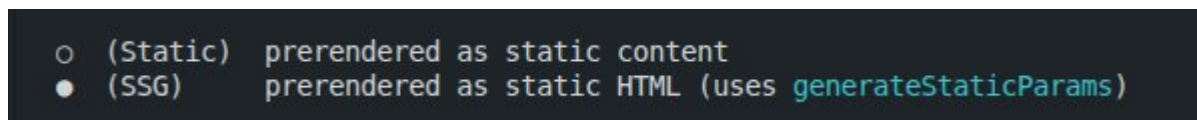


Рисунок 7 – Результат збірки проєкту з пререндерингом сторінок 1-10.

Проект було успішно збілджено командою `npm run build` та запущено у двох режимах:

- Динамічний запуск: через `npm run start` (на порту 3005 через конфлікт портів), що підтвердило працездатність серверних компонентів.
- Статичний запуск(рис.8): було використано налаштування `output: 'export'` у `next.config.mjs`. Отриману папку `out` було запущено за допомогою пакету `http-server`, що підтвердило автономність статично згенерованих сторінок.

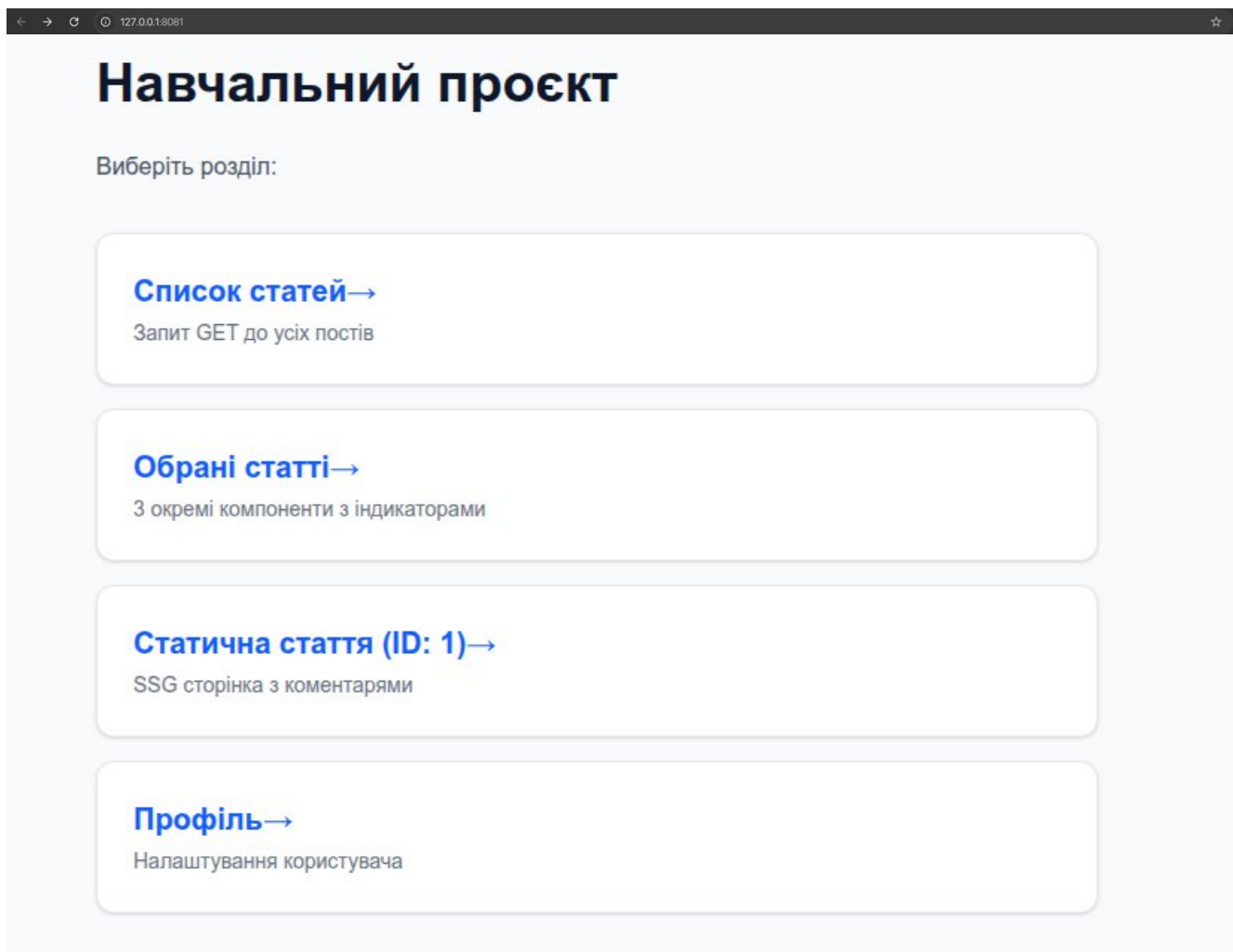


Рисунок 8 - Запуск статичної версії застосунку через http-server.

Завдання 4: Стилiзація

- Глобальні стилі: Створено структуру CSS-файлів з використанням CSS-змінних (:root) для забезпечення консистентності кольорів та шрифтів по всьому проєкту.
- Модульний CSS: Для компонента навігації використано Menu.module.css, що гарантує ізоляцію стилів та запобігає конфліктам класів.
- Кастомізація Tailwind(рис.9): Оновлено файл tailwind.config.ts, де додано нові брейкпоїнти (tablet, desktop тощо) та фірмову палітру кольорів (brand-blue).
- Інтеграція MUI: В проєкт інтегровано бібліотеку Material UI. Компоненти Button, Card та Typography використано для створення сучасного інтерфейсу, поєднуючи готові рішення бібліотеки з кастомними Tailwind-класами.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.1	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

Лістинг коду кастомізації Tailwind конфігурації:

```
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./src/pages/**/*..{js,ts,jsx,tsx,mdx}",
    "./src/components/**/*..{js,ts,jsx,tsx,mdx}",
    "./src/app/**/*..{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {

      screens: {
        'tablet': '768px',
        'laptop': '1024px',
        'desktop': '1440px',
        'ultra': '1920px',
      },

      colors: {
        'brand-blue': '#0052FF',
        'brand-dark': '#121212',
        'success-green': '#00C853',
      },
    },
  },
  plugins: [],
};

export default config;
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.1	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

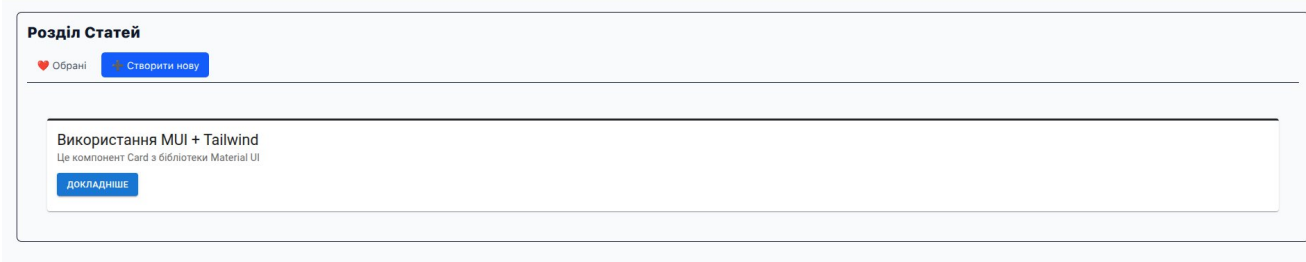


Рисунок 9 – Сторінка з використанням MUI компонентів та модульних стилів.

Завдання 5: Дизайн(рис.10)

- Колірна схема: Впроваджено єдиний стиль Taxi Style: основний чорний (#1e1e1e), акцентний жовтий (#ffcc00) та світло-сірий фон для контенту.
- Адаптивність: Додано кастомні медіа-запити (tablet:, laptop:). На малих екранах сайдбар переходить у верхній блок, а сітка статей перемикається з 2-3 колонок у одну.
- Стилiзація компонентів: * Картки: Додано товсті рамки (border-4), гострі кути (rounded-none) та "жорсткі" тіні (shadow-[10px_10px_...]) у стилі нео-бруталізму.
- Навігація: Впроваджено Hover-ефекти зі зміною кольору фону та трансформацією елементів (scale, translate).
- Інтеграція UI-бібліотек: Компоненти MUI (Card, TextField, Button) повністю перестилізовано через Tailwind CSS для відповідності загальній айденітиці проєкту.
- Типографіка: Встановлено жирне накреслення (font-black) та верхній регістр (uppercase) для всіх заголовків та елементів керування.

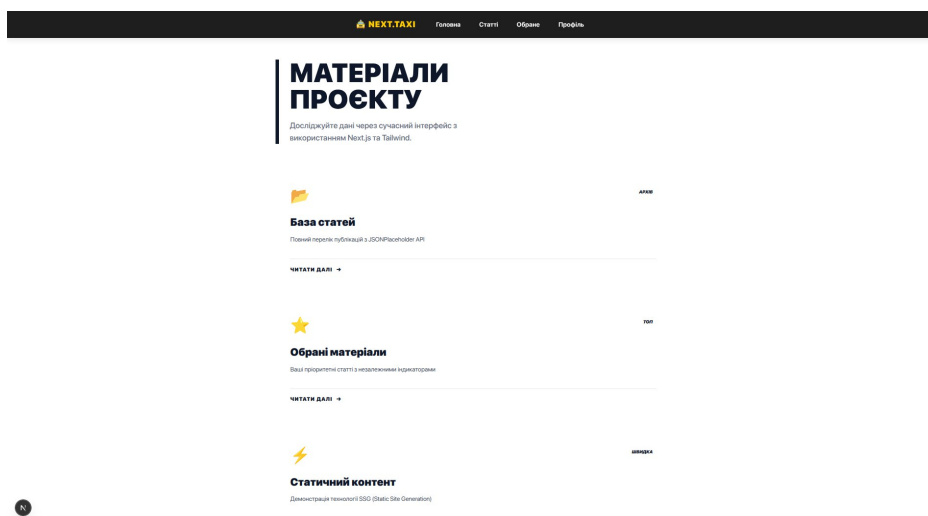


Рисунок 10 – Оновлений дизайн.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.1	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

Висновки: Під час виконання роботи я опанував повний цикл розробки сучасного вебдодатка на Next.js, навчившись ефективно поєднувати серверний та статичний рендеринг (SSG) із запитам до зовнішніх API. Я реалізував складну архітектуру вкладених лейаутів, розробив адаптивний інтерфейс у стилі Neobrutalism (Taxi Style) за допомогою Tailwind CSS та інтегрував бібліотеку MUI для створення функціональних форм. Особливу увагу було приділено оптимізації користувацького досвіду через впровадження незалежних індикаторів завантаження та трансформацію навігації у Sidebar, що забезпечило високу ергономічність та коректне відображення проєкту на будь-яких типах пристроїв.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.1	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		8